Zaki Daouk

10/25/2026

Lab 8 - C1: Numerical Agreement Test

Physics 311 | JavaScript and Lab 7 Python comparison

Purpose

This test checks that the browser implementation reproduces the Lab 7 Python projectile simulation when both use the same initial state, gravitational acceleration, integrator, timestep, and number of steps.

Simulation setup

System	**One projectile; mass = 1 kg; initial position = (0, 0) m.**
Initial velocity	20 m/s at 50 degrees above horizontal.
Force	Uniform gravity, g = 9.81 m/s^2 downward; no drag.
Integrator	Euler-Cromer in both Python and JavaScript.
Timestep	dt = 1/120 s.
Run length	1,200 steps, corresponding to 10 s of simulated time.

Final-state comparison

The JavaScript values were printed by the Lab 8 Run 10 s headless (C1) button. The Python values were obtained by running the same setup through the Lab 7 engine. The absolute difference column is computed from the displayed full-precision values.

Quantity	Python (Lab 7)	JavaScript (Lab 8)	|difference|
x (m)	128.55752193730959	128.55752193730959	0
y (m)	-337.69986137620293	-337.69986137620293	0
vx (m/s)	12.855752193730787	12.855752193730787	0
vy (m/s)	-82.779111137620035	-82.779111137620035	0
E_total (J)	195.99016249998067	195.99016249998067	0

Conclusion

The Python and JavaScript final states agree at the displayed precision for all five quantities. This confirms that the two implementations used matching initial conditions, Euler-Cromer updates, timestep, and step count for this test. The zero differences shown are differences between the displayed values; any remaining discrepancy below the printed precision is at floating-point round-off scale.

Screenshot

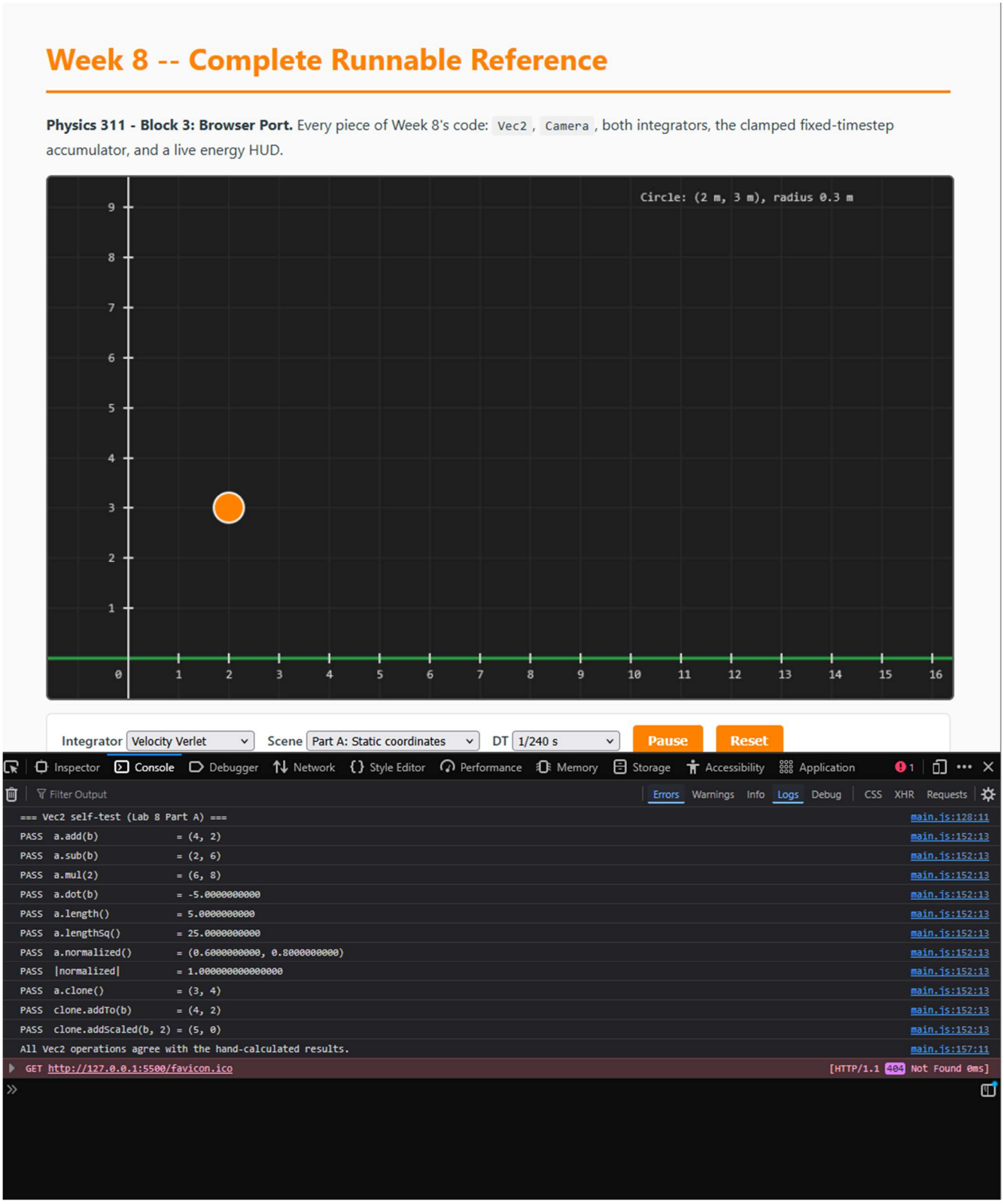


Fig. 1 Final State and Console Print Showing Logs

Lab 8 - C2: Fixed vs. Variable Timestep

Physics 311 | Lag demonstration using the spring oscillator

Test setup

System	**Spring oscillator, m = 1 kg and k = 1 N/m.**
Integrator	Velocity Verlet.
Nominal fixed step	DT = 1/20 s = 0.05 s.
Lag applied	The C2 control blocked the browser main thread for 500 ms.

Recorded observations

The app induced a 500 ms main-thread delay. Fixed mode capped catch-up at 0.25 s; variable mode advanced by the entire measured frame duration. The screenshots preserve each mode and its post-lag HUD state, with the lag-frame measurements labeled below.

Mode	Lag frame dt	Simulated-time advance	Steps	E_total before (J)	E_total after (J)
Fixed accumulator	0.5065 s	0.2500 s (clamped)	5 x 0.05 s	0.499728506	0.499692026
Variable frame dt	0.5063 s	0.5063 s	1 x 0.5063 s	0.499968765	0.497995586

Observation and explanation

I tested the m = k = 1 spring oscillator with Velocity Verlet and nominal DT = 0.05 s, then introduced a 500 ms main-thread delay. In fixed mode, the measured lag frame was 0.5065 s; the accumulator advanced only 0.25 s in five 0.05 s steps, and E_total changed from 0.499728506 J to 0.499692026 J. In variable mode, the measured lag frame was 0.5063 s and the simulation took one 0.5063 s step; E_total changed from 0.499968765 J to 0.497995586 J. The post-lag HUDs show different trajectories: fixed mode at t = 1.550 s, x = 0.021 m, versus variable mode at t = 1.829 s, x = -0.276 m. The variable-step energy change was much larger, illustrating why symplectic Verlet relies on a constant timestep: variable frame durations break its fixed-step structure and increase energy drift.

Screenshots

The labeled evidence images below show the canvas, active timestep mode, and post-lag trajectory state; the annotations record the measured lag-frame results.

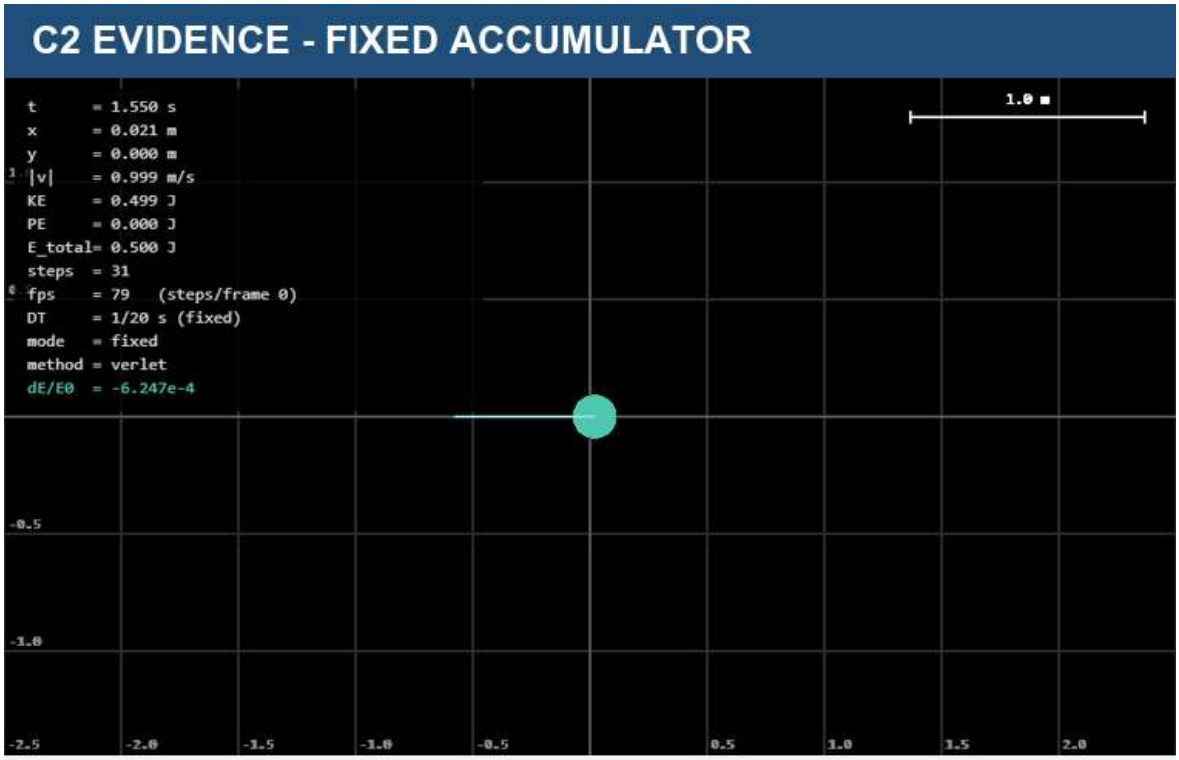


Figure 1. Fixed accumulator after an induced lag spike

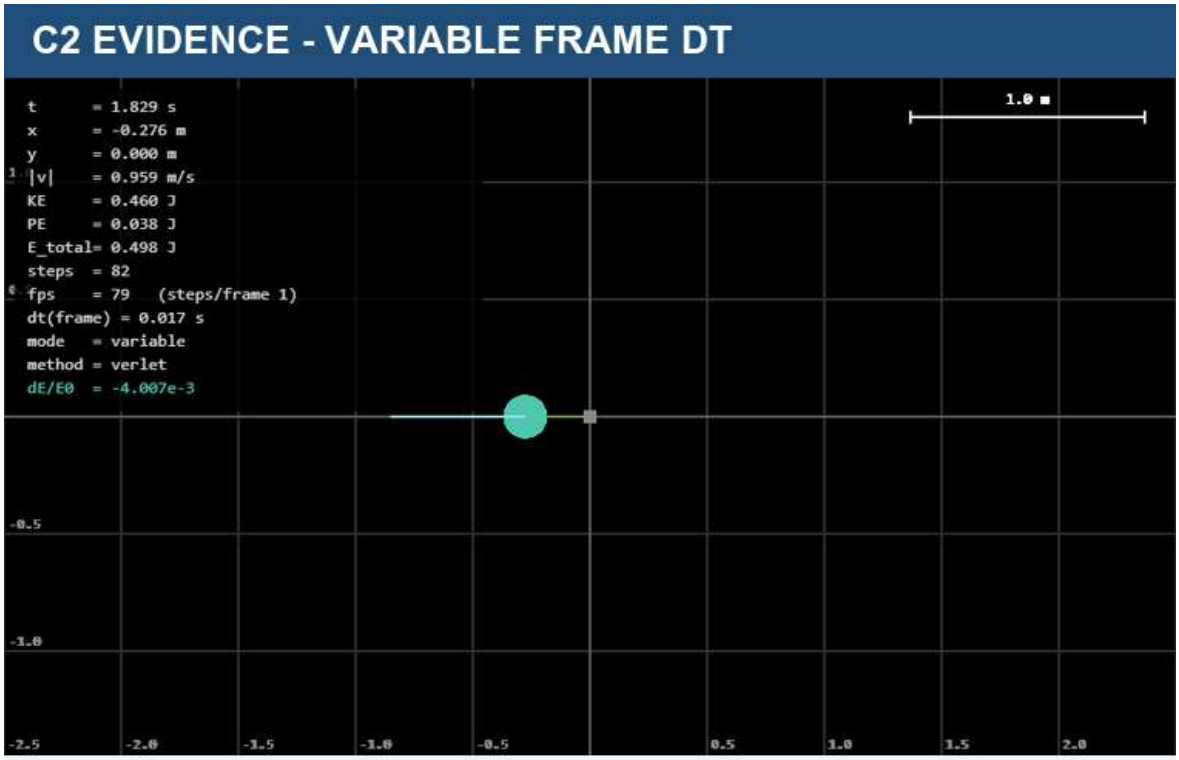


Figure 2. Variable frame timestep after an induced lag spike

Lab 8 - C3: Live Integrator Comparison

Physics 311 | Runtime Euler-Cromer vs. Velocity Verlet

Live control and test setup

The Integrator (live switch) dropdown changes the active method immediately without resetting the current simulation. For a controlled comparison, I used the spring oscillator (m = k = 1, x0 = 1 m, v0 = 0), fixed DT = 1/20 s, and 200 steps (10.0 s) for each method, starting from the same initial state.

Total energy comparison

Integrator	Steps / duration	E_total initial (J)	E_total final (J)	HUD final (3 d.p.)
Euler-Cromer	200 / 10.0 s	0.500000	0.488759693	0.489 J
Velocity Verlet	200 / 10.0 s	0.500000	0.499907215	0.500 J

With this coarse timestep, Euler-Cromer's final energy is about 0.01124 J below the initial value, whereas Velocity Verlet remains within about 0.000093 J; the live HUD therefore reads 0.489 J versus 0.500 J at 10 s.

Why the projectile is a weak comparison

The ideal projectile has constant acceleration, so Euler-Cromer and Verlet produce the same velocity update; its position difference is a predictable O(DT) offset rather than a demanding test of how a method handles changing forces. The Week 2, Section 5.5 closed-form result lets you predict that offset, whereas the position-dependent restoring force in the Block 2 Lab 3 simple harmonic oscillator makes phase and energy errors accumulate and separates the methods much more dramatically.

-

The trickiest change was adapting the simulation loop to the browser. Python can step through a simulation directly, but JavaScript animation runs on browser frames, whose timing can vary. I used a fixed-timestep accumulator and checked it by comparing the numerical results and then inducing lag to observe the energy change.

The Bonus link is:

https://35mmfx.github.io/phys311/labs/lab08/

It seems to work fine on my iphone with the duck duck go browser. The type is harder to see thought due to its size.